

多头履约行为查询服务（dtly）· API 接口文档与使用手册

面向接入方（客户）技术与运维人员的对外接口说明。

版本：dtly（多头履约行为） | 通信：HTTP + JSON | 编码：UTF-8

本服务采用统一请求信封（`appKey` / `sign` / `encryptionType` / `body`）与统一响应结构（`head` + `body`）。调用方对 `body` 业务参数按 MD5 规则加签即可。

本接口入参为个人三要素 `name`、`idCard`、`mobile`，三项均必填，缺任一项或格式非法将被直接拒绝且不计费。

查询结果为结构化对象，本服务将其序列化为 JSON 字符串置于 `body.result.range`，调用方需二次解析（见 3.1.5）。该对象包含 357 个履约行为因子与评分字段。

计费提示：本服务按有效查询次数计费——查得数据（001）与查无结果（999）均计费，详见第五章。

授权提示：调用前须取得被查询人的合法授权；授权书**不通过接口传输**，由调用方经商务渠道提交我方，详见 1.5。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「多头履约行为查询服务（dtly）」的第三方产品技术开发人员与日常运维人员。

1.2 接入须知

1. 正式服务地址（任选其一，两域名等价）：`http://aiszcloud.cn:8080`、`http://aiszcloud.com.cn:8080`。
2. 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret`（加签密钥）。
3. 本服务按有效查询次数计费（查得与查无均计费），请做好查询频次控制，避免短时间内对同一主体重复发起调用。
4. 调用前须按 1.5 的要求准备并提交被查询人授权书。

1.3 接口说明

项目	说明
请求方式	POST（计费查询数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（建议客户端读超时 ≥ 10 秒）
HTTP 状态码	恒为 200；业务结果与错误均通过响应体的 <code>head.errorCode</code> / <code>body.code</code> 表达
签名	调用时需对 <code>body</code> 中的业务参数 + 我方分配的 <code>appSecret</code> 进行 MD5 加签，详见第二章

1.4 环境说明

- **正式环境**：使用正式账户，调用已开通接口，按有效查询次数计费（见第五章）。
- **测试环境**：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

1.5 授权要求

1. 调用本接口查询任何个人信息前，调用方**必须已取得被查询人合法、有效的授权**（个人信息查询授权书）。
2. 授权书**不通过 API 传输**：请求中无需、也不应携带任何授权文件、授权链接或授权类型字段，本接口的请求参数仅为 3.1.1 所列三要素。
3. 授权书由调用方**通过商务渠道提交给我方**，具体提交方式、文件格式与提交频次由商务在开通账户时与调用方约定；我方收到后进行审核、留存等后续处理。
4. 请在正式调用前与商务确认授权书提交方式并完成首次提交。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y890dtly	String	是	我方分配给客户的公开标识
sign	7388c24a8546864b6c6589c86e8ee816	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	参数加密类型，1 = 明文（默认）
body	{...}	Object	是	业务请求体，见各接口定义

注意：本服务不需要额外的产品编号参数（产品由请求路径区分）。appKey / sign / encryptionType 不参与签名计算。
调用方传入的 body 字段一律为明文（姓名 / 身份证号 / 手机号）。

2.2 鉴权校验顺序

本服务按以下顺序校验，任一失败立即返回对应 head.errorCode（参数非法直接拒绝，不计费）：

- 1. appKey 是否存在 → 否则 505001
- 2. appKey 是否匹配到账户 → 否则 505004
- 3. 账户是否有效（启用且在有效期内）→ 否则 505007
- 4. 签名是否正确 → 否则 505002

2.3 加签方式

- 1. 取出 body 中所有非空的业务参数（不含文件/字节流类型，不含值为空的参数）。
- 2. 按参数名（key）的 ASCII 升序排序；首字符相同则依次比较后续字符。
- 3. 将排序后的参数按 参数名 参数值 直接拼接，最后追加 appSecret，得到待签名串。
- 4. 对待签名串做 MD5，取 32 位小写十六进制字符串，赋值给 sign。

示例：body = { "name": "张三", "idCard": "11010519491231002X", "mobile": "13809091009" } , appSecret = <你的密钥>。

按 ASCII 升序排序后 key 顺序为 idCard → mobile → name，待签名串为：

```
idCard11010519491231002Xmobile13809091009name张三<你的密钥>
```

sign = MD5(待签名串) 的小写十六进制值。

提示：拼接顺序由 key 的 ASCII 决定，请勿写死字段顺序。加签的范围是你实际发送的 body 全部非空字段——若额外传了本接口不需要的字段，它同样要参与加签，否则会被判 505002。

2.4 加签代码示例

Java

```
public static String sign(Map<String, String> bodyParams, String appSecret) throws Exception {
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```
import hashlib

def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()): # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest() # 小写 hex
```

Go

```
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

三、接口列表

3.1 多头履约行为查询 (dtly)

项目	内容
路径	POST /v1/openapi/zlx/querySrmxDTLY
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/querySrmxDTLY (或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/querySrmxDTLY)
鉴权	appKey + MD5 签名 (见第二章)

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
name	张三	String	是	姓名, 明文传入
idCard	11010519491231002X	String	是	身份证号 (18 位, 末位可为 x), 须为校验位合法的有效号码, 明文传入
mobile	13809091009	String	是	手机号 (11 位, 以 1 开头), 明文传入

三项均必填。 任一字段缺失或格式非法 (身份证号非 18 位规范号码、手机号非 11 位) 将返回

head.errorCode = 505062 , 直接拒绝, 不计费。

身份证号仅满足 18 位格式但校验位不正确时, 也可能返回 505062 (服务处理异常), 请使用真实有效

的身份证号联调。

本接口查询须取得被查询人的合法授权。**授权书不在请求中携带**，由调用方通过商务渠道提交我方，我方进行后续处理（见 1.5）。

3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y890dtly",
  "sign": "<按 2.3 规则计算>",
  "body": {
    "name": "张三",
    "idCard": "11010519491231002X",
    "mobile": "13809091009"
  }
}
```

3.1.3 响应结构

响应分为 head（网关头部）与 body（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。0 = 成功（含查得/查无）；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时需提供
time	125	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 head.errorCode = 0 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。001 = 查得数据【计费】；999 = 查无结果【计费】
msg	成功	String	业务描述
reqid	1kf9x2...	String	本次请求流水号
uid	lgt17901412838070	String	交易流水号（对账用）
result	{...}	Object	业务内容，仅 code = 001 时存在
result.range	"{...}"	String	履约行为结果的 JSON 字符串（对象），需二次解析，结构见 3.1.5

3.1.4 响应示例

1) 查得数据（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 125,
"timestamp": 1718456789012 },
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "dkykagq5m17u3",
    "uid": "lgt17901412838070",
    "result": {
      "range": "{\"xyp_cpl0001\": \"2\", \"xyp_cpl0044\": \"0\", \"xyp_model_score_high\": \"612\", \"xyp_model_score_mid\": \"-1\", \"xyp_model_score_low\": \"-1\", \"xyp_t01aazhzz\": \"\", \"xyp_var1\": \"\"}"
    }
  }
}
```

（示例 range 仅节选部分字段，实际返回包含全部 357 个字段。）

2) 查无结果（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d5", "time": 96, "timestamp": 1718456789013 },
  "body": {
    "code": "999",
    "msg": "查无结果",
    "reqid": "lkf9x2ac",
    "uid": "lgt17901412839871"
  }
}
```

999 表示本次查询已完成、未查得该主体的相关记录——这是一个有效的查询结论，与 001 同样计费（见第五章）。

3) 网关级错误 (无 body)

```
{
  "head": { "errorCode": "505062", "errorMsg": "数据请求异常", "logId": "a1b2c3d6", "time": 12, "timestamp": 1718456789014 }
}
```

3.1.5 result.range 结构说明

result.range 是履约行为结果序列化后的 JSON 字符串。调用方应先对该字符串做一次 JSON.parse / 反序列化，得到一个 JSON 对象（非数组）。

对象内共 357 个字段，值均为 String 类型，分为四类：

类别	字段前缀	数量	说明
评分	xyp_model_score_*	3	履约风险评分，见下「一、评分字段」
履约行为因子	xyp_cp1*	78	业务含义明确的汇总因子，见下「二、履约行为因子」
分层明细因子	xyp_t01* / xyp_t02* / xyp_t03* / xyp_t04*	266	编码型明细因子，见下「三、分层明细因子」
预留	xyp_var1 ~ xyp_var10	10	预留字段，当前恒为空串

一、评分字段

字段	含义	取值
xyp_model_score_high	小额网贷评分	[350,950]，分值越大逾期率越低；未命中为 -1
xyp_model_score_mid	小额分期评分	[350,950]，分值越大逾期率越低；未命中为 -1
xyp_model_score_low	中大额分期评分	[350,950]，分值越大逾期率越低；未命中为 -1

⚠ -1 表示该主体未命中该评分模型，不是分数为负，也不代表查无；请勿把 -1 当作数值参与阈值比较。

二、履约行为因子（xyp_cpl*，共 78 个）

借贷机构数

字段	含义	取值
xyp_cpl0001	贷款总机构数	区间档位（见 3.1.6）
xyp_cpl0002	贷款已结清机构数	区间档位（见 3.1.6）
xyp_cpl0007	消费金融类机构数（有场景的、分期）	区间档位（见 3.1.6）
xyp_cpl0008	网络贷款类机构数（现金贷）	区间档位（见 3.1.6）
xyp_cpl0070	最近1天贷款机构数	区间档位（见 3.1.6）
xyp_cpl0009	最近7天贷款机构数	区间档位（见 3.1.6）
xyp_cpl0010	最近14天贷款机构数	区间档位（见 3.1.6）
xyp_cpl0063	最近21天贷款机构数	区间档位（见 3.1.6）
xyp_cpl0011	最近30天贷款机构数	区间档位（见 3.1.6）
xyp_cpl0012	最近90天贷款机构数	区间档位（见 3.1.6）
xyp_cpl0013	最近180天贷款机构数	区间档位（见 3.1.6）

还款成功 / 交易失败笔数

字段	含义	取值
xyp_cpl0014	历史贷款机构成功还款笔数	区间档位 (见 3.1.6)
xyp_cpl0015	历史贷款机构交易失败笔数	区间档位 (见 3.1.6)
xyp_cpl0016	最近1天交易失败笔数	区间档位 (见 3.1.6)
xyp_cpl0017	最近1天还款成功笔数	区间档位 (见 3.1.6)
xyp_cpl0018	最近7天交易失败笔数	区间档位 (见 3.1.6)
xyp_cpl0019	最近7天还款成功笔数	区间档位 (见 3.1.6)
xyp_cpl0020	最近14天交易失败笔数	区间档位 (见 3.1.6)
xyp_cpl0021	最近14天还款成功笔数	区间档位 (见 3.1.6)
xyp_cpl0064	最近21天还款成功笔数	区间档位 (见 3.1.6)
xyp_cpl0065	最近21天交易失败笔数	区间档位 (见 3.1.6)
xyp_cpl0022	最近30天交易失败笔数	区间档位 (见 3.1.6)
xyp_cpl0023	最近30天还款成功笔数	区间档位 (见 3.1.6)
xyp_cpl0024	最近90天交易失败笔数	区间档位 (见 3.1.6)
xyp_cpl0025	最近90天还款成功笔数	区间档位 (见 3.1.6)
xyp_cpl0026	最近180天交易失败笔数	区间档位 (见 3.1.6)
xyp_cpl0027	最近180天还款成功笔数	区间档位 (见 3.1.6)

还款成功 / 交易失败金额

字段	含义	取值
xyp_cpl0032	最近1天交易失败总金额	区间档位 (见 3.1.6)
xyp_cpl0033	最近1天还款成功总金额	区间档位 (见 3.1.6)
xyp_cpl0034	最近7天交易失败总金额	区间档位 (见 3.1.6)
xyp_cpl0035	最近7天还款成功总金额	区间档位 (见 3.1.6)
xyp_cpl0036	最近14天交易失败总金额	区间档位 (见 3.1.6)
xyp_cpl0037	最近14天还款成功总金额	区间档位 (见 3.1.6)
xyp_cpl0066	最近21天交易失败总金额	区间档位 (见 3.1.6)
xyp_cpl0067	最近21天还款成功总金额	区间档位 (见 3.1.6)
xyp_cpl0038	最近30天交易失败总金额	区间档位 (见 3.1.6)
xyp_cpl0039	最近30天还款成功总金额	区间档位 (见 3.1.6)
xyp_cpl0040	最近90天交易失败总金额	区间档位 (见 3.1.6)
xyp_cpl0041	最近90天还款成功总金额	区间档位 (见 3.1.6)
xyp_cpl0042	最近180天交易失败总金额	区间档位 (见 3.1.6)
xyp_cpl0043	最近180天还款成功总金额	区间档位 (见 3.1.6)

逾期标识 (是 / 否)

字段	含义	取值
xyp_cpl0044	当前是否存在逾期未结清	1 = 是 / 0 = 否
xyp_cpl0028	最近1天是否发生过逾期	1 = 是 / 0 = 否
xyp_cpl0029	最近7天是否发生过逾期	1 = 是 / 0 = 否
xyp_cpl0030	最近14天是否发生过逾期	1 = 是 / 0 = 否
xyp_cpl0031	最近30天是否发生过逾期	1 = 是 / 0 = 否

当前逾期与失败机构数

字段	含义	取值
xyp_cpl0071	当前逾期机构数	区间档位 (见 3.1.6)
xyp_cpl0072	当前逾期金额	区间档位 (见 3.1.6)
xyp_cpl0048	最近7天_交易失败次数>0的机构数	区间档位 (见 3.1.6)
xyp_cpl0049	最近14天_交易失败次数>0的机构数	区间档位 (见 3.1.6)
xyp_cpl0050	最近21天_交易失败次数>0的机构数	区间档位 (见 3.1.6)
xyp_cpl0051	最近30天_交易失败次数>0的机构数	区间档位 (见 3.1.6)

时长与间隔 (单位: 天)

字段	含义	取值
xyp_cpl0045	信用贷款时长	区间档位 (见 3.1.6)
xyp_cpl0046	最近一次交易距离当前时间	区间档位 (见 3.1.6)
xyp_cpl0068	最近一次还款成功距离当前天数	区间档位 (见 3.1.6)

近 N 次还款表现

字段	含义	取值
xyp_cpl0073	近5次还款中成功还款总金额/近5次还款总金额	比值, 0 ~ 1 之间
xyp_cpl0074	近5次还款中还款成功笔数/近5次还款总笔数	比值, 0 ~ 1 之间
xyp_cpl0075	近20次还款中小贷担保类还款成功笔数/近20次还款中小贷担保类还款笔数	比值, 0 ~ 1 之间
xyp_cpl0076	近20次还款中还款成功笔数	区间档位 (见 3.1.6)
xyp_cpl0077	近5次还款中还款成功总金额	区间档位 (见 3.1.6)
xyp_cpl0078	近20次还款中还款成功总金额	区间档位 (见 3.1.6)
xyp_cpl0079	近90天内还款成功总金额/近90天还款总金额	比值, 0 ~ 1 之间
xyp_cpl0080	近90天内还款成功笔数/近90天内还款总笔数	比值, 0 ~ 1 之间

交易失败后的恢复表现

字段	含义	取值
xyp_cpl0052	消费金融类最后一次交易失败后还款次数	区间档位 (见 3.1.6)
xyp_cpl0053	小贷担保类最后一次交易失败后还款次数	区间档位 (见 3.1.6)
xyp_cpl0069	最后一次交易失败后还款次数	区间档位 (见 3.1.6)
xyp_cpl0054	消费金融类_交易失败向后距离下一次还款成功的天数最大值	区间档位 (见 3.1.6)
xyp_cpl0055	小贷担保类_交易失败向后距离下一次还款成功的天数最大值	区间档位 (见 3.1.6)
xyp_cpl0056	交易失败向后距离下一次还款成功的天数最大值	区间档位 (见 3.1.6)
xyp_cpl0057	消费金融类_交易失败向后距离下一次还款成功的天数最小值	区间档位 (见 3.1.6)
xyp_cpl0058	小贷担保类_交易失败向后距离下一次还款成功的天数最小值	区间档位 (见 3.1.6)
xyp_cpl0059	交易失败向后距离下一次还款成功的天数最小值	区间档位 (见 3.1.6)
xyp_cpl0060	消费金融类_交易失败向后距离下一次还款成功的天数平均值	区间档位 (见 3.1.6)
xyp_cpl0061	小贷担保类_交易失败向后距离下一次还款成功的天数平均值	区间档位 (见 3.1.6)
xyp_cpl0062	交易失败向后距离下一次还款成功的天数平均值	区间档位 (见 3.1.6)

综合指数

字段	含义	取值
xyp_cpl0081	信用风险评分	0 ~ 1 之间, 分数越高用户信用越低
xyp_cpl0082	履约金额综合指数	0 ~ 1 之间, 指数越大用户逾期可能性越高
xyp_cpl0083	履约笔数综合指数	0 ~ 1 之间, 指数越大用户逾期可能性越高

三、分层明细因子 (266 个)

xyp_t01* / xyp_t02* / xyp_t03* / xyp_t04* 是编码型的分层履约明细因子 (如 xyp_t01aazhzz、xyp_t02cchzza_cchzzz、xyp_t03td148、xyp_t0400002), 字段名本身为内部编码, 不承载可读语义, 取值口径与履约行为因子一致 (多数为区间档位)。本服务对这批字段原样透出、不裁剪、不改写字段名, 供有建模需求的调用方直接入模使用; 无建模需求的调用方可只取评分字段与 xyp_cpl* 因子。

四、取值口径 (重要)

- 绝大多数因子输出的是「区间档位」而非原始数值。例如 xyp_cpl0001 (贷款总机构数) 返回 "2" 表示落在 [9,14) 这一档, 不是 2 家机构。逐字段的档位区间见 3.1.6。
- 全体字段通用的三个特殊取值:

取值	含义
"0"	原值为 0
""（空串）	该因子缺失（无相关记录）
"-1"	原值为负数

- 少数字段是「是/否」语义（"1" = 是 / "0" = 否），已在上表「取值」列逐字段标注。
- 少数字段是 0 ~ 1 之间的比值或指数，直接为数值文本，无需查档位表。

说明：`result.range` 的具体字段以实际返回为准，本服务保证原样透出、不裁剪、不改写字段名。因子集合可能随数据丰富度变化，建议调用方按「存在即解析、缺失即忽略」做容错处理，不要假设某个因子键一定存在。

⚠ 重要：因子大量为空串、评分为 -1 并不代表查无。 只要返回 `body.code = 001`，就说明本次已取得该主体的明确履约行为结论——「无相关记录」本身就是一个有效结论（属查得，计费）。
`body.code = 999` 才是查无（未查得该主体的相关记录，同样计费）。

解析示例（JavaScript）：

```
const data = JSON.parse(resp.body.result.range);

// 1) 评分：-1 表示未命中模型，必须先排除再比较
const score = data.xyp_model_score_high;
if (score && score !== "-1") {
  console.log("小额网贷评分:", Number(score));
} else {
  console.log("未命中该评分模型");
}

// 2) 「是/否」类因子
if (data.xyp_cpl0044 === "1") {
  console.log("当前存在逾期未结清");
}

// 3) 区间档位因子：拿到的是档位号，需按 3.1.6 的对照表解释
const tier = data.xyp_cpl0001; // 如 "2"
if (tier === "") {
  console.log("贷款总机构数：无记录");
} else {
  console.log("贷款总机构数档位:", tier);
}

// 4) 按需遍历，缺失即忽略
Object.keys(data)
  .filter((k) => k.startsWith("xyp_cpl") && data[k] !== "")
  .forEach((k) => console.log(k, "=", data[k]));
```

3.1.6 区间档位对照表

下表给出「履约行为因子」中采用区间档位输出的字段与其档位含义。区间记法为数学区间： (a,b) 不含端点， $[a,b)$ 含左不含右， Inf 表示无上界。金额类字段的区间单位为元，时长类字段的区间单位为天。

未在本表出现的字段为「是/否」字段或 $0 \sim 1$ 比值/指数字段，直接按值解释即可。

字段	档位 → 区间
xyp_cpl0001	2 = [9,14); 3 = [14, Inf)
xyp_cpl0002	2 = [5,9); 3 = [9,14); 4 = [14,17); 5 = [17,Inf)
xyp_cpl0007	2 = [1,2); 3 = [2,4); 4 = [4, Inf)
xyp_cpl0008	2 = [1,5); 3 = [5,11); 4 = [11, Inf)
xyp_cpl0070	2 = [1,3); 3 = [3, Inf)
xyp_cpl0009	2 = [1,9); 3 = [9, Inf)
xyp_cpl0010	2 = [1,3); 3 = [3,10); 4 = [10, Inf)
xyp_cpl0063	2 = [3,6); 3 = [6,11); 4 = [11, Inf)
xyp_cpl0011	2 = [1,4); 3 = [4,11); 4 = [11, Inf)
xyp_cpl0012	2 = [5,10); 3 = [10,17); 4 = [17, Inf)
xyp_cpl0013	2 = [4,7); 3 = [7,13); 4 = [13, Inf)
xyp_cpl0014	2 = [2,10); 3 = [10,34); 4 = [34,60); 5 = [60, Inf)
xyp_cpl0015	2 = [3,5); 3 = [5,10); 4 = [10,Inf)
xyp_cpl0016	2 = [3,5); 3 = [5,Inf)
xyp_cpl0017	2 = [3, Inf)
xyp_cpl0018	2 = [3,5); 3 = [5,7); 4 = [7,Inf)
xyp_cpl0019	2 = [3,13); 3 = [13, Inf)
xyp_cpl0020	2 = [3,5); 3 = [5,15); 4 = [15,Inf)
xyp_cpl0021	2 = [5,21); 3 = [21, Inf)
xyp_cpl0064	2 = [5,27); 3 = [27, Inf)
xyp_cpl0065	2 = [3,5); 3 = [5,20); 4 = [20,Inf)
xyp_cpl0022	2 = [3,5); 3 = [5,34); 4 = [34,Inf)
xyp_cpl0023	2 = [7,34); 3 = [34, Inf)
xyp_cpl0024	2 = [6,22); 3 = [22,56); 4 = [56,Inf)
xyp_cpl0025	2 = [2,31); 3 = [31, Inf)
xyp_cpl0026	2 = [3,25); 3 = [25,30); 4 = [30,70); 5 = [70,Inf)
xyp_cpl0027	2 = [3,27); 3 = [27,50); 4 = [50, Inf)
xyp_cpl0032	2 = [1500,2500); 3 = [2500,5000); 4 = [5000,Inf)

字段	档位 → 区间
xyp_cpl0033	2 = [1500,5000); 3 = [5000, Inf)
xyp_cpl0034	2 = [2000,10000); 3 = [10000,17000); 4 = [17000,26000); 5 = [26000,Inf)
xyp_cpl0035	2 = [2000,17000); 3 = [17000, Inf)
xyp_cpl0036	2 = [1000,4000); 3 = [4000,9000); 4 = [9000,30000); 5 = [30000,Inf)
xyp_cpl0037	2 = [9000,31000); 3 = [31000, Inf)
xyp_cpl0066	2 = [1000,2000); 3 = [2000,30000); 4 = [30000,50000); 5 = [50000,Inf)
xyp_cpl0067	2 = [1000,34000); 3 = [34000, Inf)
xyp_cpl0038	2 = [6000,10000); 3 = [10000,50000); 4 = [50000,Inf)
xyp_cpl0039	2 = [10000,36000); 3 = [36000, Inf)
xyp_cpl0040	2 = [10000,50000); 3 = [50000,80000); 4 = [80000,Inf)
xyp_cpl0041	2 = [13000,49000); 3 = [49000, Inf)
xyp_cpl0042	2 = [2000,30000); 3 = [30000,60000); 4 = [60000,90000); 5 = [90000,Inf)
xyp_cpl0043	2 = [2000,58000); 3 = [58000, Inf)
xyp_cpl0045	2 = [220,450); 3 = [450,520); 4 = [520, Inf)
xyp_cpl0046	2 = [5,25); 3 = [25, Inf)
xyp_cpl0048	2 = [1,2); 3 = [2, Inf)
xyp_cpl0049	2 = [1,3); 3 = [3, Inf)
xyp_cpl0050	2 = [1,3); 3 = [3, Inf)
xyp_cpl0051	2 = [1,3); 3 = [3, Inf)
xyp_cpl0068	2 = [5,50); 3 = [50,160); 4 = [160, Inf)
xyp_cpl0071	2 = [2,4); 3 = [4, Inf)
xyp_cpl0072	2 = [1000,2000); 3 = [2000,3000); 4 = [3000,5000); 5 = [5000,7000); 6 = [7000,11000); 7 = [11000,Inf)
xyp_cpl0076	2 = [2,11); 3 = [11,13); 4 = [13, Inf)
xyp_cpl0077	2 = [4000,8000); 3 = [8000, Inf)
xyp_cpl0078	2 = [2000,25000); 3 = [25000, Inf)
xyp_cpl0052	2 = [1,2); 3 = [2,4); 4 = [4, Inf)
xyp_cpl0053	2 = [1,2); 3 = [2,4); 4 = [4,10); 5 = [10, Inf)

字段	档位 → 区间
xyp_cpl0069	2 = [2,4); 3 = [4,10); 4 = [10, Inf)
xyp_cpl0054	2 = [14, Inf)
xyp_cpl0055	2 = (3,5]; 3 = (5,8]; 4 = (8,10]; 5 = (10,15]; 6 = (15,20]; 7 = (20,30]; 8 = (30,50]; 9 = (50,100]; 10 = (100,200]; 11 = (200,500]; 12 = (500,1000]; 13 = (1000,Inf)
xyp_cpl0056	2 = [5,20); 3 = [20,35); 4 = [35, Inf)
xyp_cpl0057	2 = [1, Inf)
xyp_cpl0058	2 = (3,5]; 3 = (5,8]; 4 = (8,10]; 5 = (10,15]; 6 = (15,20]; 7 = (20,30]; 8 = (30,50]; 9 = (50,100]; 10 = (100,200]; 11 = (200,500]; 12 = (500,1000]; 13 = (1000,Inf)
xyp_cpl0059	2 = [10, Inf)
xyp_cpl0060	2 = [5, Inf)
xyp_cpl0061	2 = (3,5]; 3 = (5,8]; 4 = (8,10]; 5 = (10,15]; 6 = (15,20]; 7 = (20,30]; 8 = (30,50]; 9 = (50,100]; 10 = (100,200]; 11 = (200,500]; 12 = (500,1000]; 13 = (1000,Inf)
xyp_cpl0062	2 = [1,2); 3 = [2,36); 4 = [36, Inf)

3.2 计费查询数查询（扩展接口）

查询本账户累计计费查询的次数与累计调用次数，用于客户侧自助监控与对账。不返回额度上限/剩余量（本服务无额度限制）。

项目	内容
路径	GET /v1/openapi/zlx/quotaDTLY
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/quotaDTLY （或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/quotaDTLY ）
鉴权	同 3.1 主查询（请求体携带 appKey + sign 信封；body 可为 {}，此时 sign = MD5(appSecret)）

响应示例：

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1280
}
```

参数	说明
status	账户状态（ACTIVE / SUSPENDED 等）
serviceUsed	累计计费查询次数（code = 001 与 code = 999 均计入）
totalCalls	累计有效查询次数（含查得与查无；不含被网关拦截的请求）

说明：无任何额度上限拦截，仅做计费次数统计。本服务对查得与查无同等计费，故 serviceUsed 与 totalCalls 在正常情况下一致；对账请以 serviceUsed 为准。

3.3 健康检查

项目	内容
路径	GET /healthz
完整地址	http://aiszcloud.cn:8080/healthz（或 http://aiszcloud.com.cn:8080/healthz）
鉴权	无
响应	HTTP 200，纯文本 ok

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功（业务结果见 body.code）
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505003	产品编号异常	保留
505062	数据请求异常	参数非法 / 服务处理异常 / 超时未决 / 系统错误（默认错误码）

服务侧的账户/参数/系统等异常均归一为网关 505062，不计费，并经异步复查兜底。

4.2 业务结果码 body.code (仅 errorCode = 0 时)

code	含义	是否计费
001	查得数据	计费
999	查无结果	计费

注：本服务按有效查询次数计费。只要本次查询产出了确定结论——无论是查得数据（001，即使大量因子为空串、评分为 -1）还是确认无相关记录（999）——均计费一次。仅当返回 head.errorCode 非 0（未能产出结论）时不计费。

五、计费说明

body.code = 001（查得数据，含「因子大量为空、评分未命中」这一结论）与 body.code = 999（查无结果）均计入计费查询次数并对客户计费。本服务按有效查询次数计费：查无结果同样消耗一次查询并产出确定结论。网关级错误（head.errorCode 非 0：鉴权失败、参数非法、服务处理异常、系统异常等）一律不计费。计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。计费次数可随时通过计费查询数查询（3.2）接口自助核对。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 appKey、appSecret；服务地址见 1.2 接入须知。
2. 与商务确认被查询人授权书的提交方式，并按约定通过商务渠道提交授权书（见 1.5）；授权书不通过接口传输。
3. 按第二章实现加签，先在测试环境联调，再切正式环境。
4. 上线后通过计费查询数查询接口（3.2）监控调用量与计费次数。

6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（≥ 10 秒）。
- 收到网络超时/无响应时可安全重试：本服务基于内部流水号做幂等，不会因重试重复计费。
- 请勿对已明确返回 head.errorCode 的请求做无差别重试（如参数错误 505062、鉴权错误 505001 / 505002 / 505004），应先修正再发起。

6.3 错误处理建议

现象	排查方向
505001 / 505004	检查 appKey 是否正确、是否用错环境账户
505002	检查签名算法（排序/空值剔除/UTF-8/小写 hex）；确认实际发送的每一个 body 字段都参与了加签
505007	联系商务确认账户状态与有效期
505062	检查 name、idCard、mobile 是否三项齐全，身份证号是否有效（含校验位）、手机号是否为 11 位；若入参正常仍持续出现，记录 logId 联系我方
因子大量为空串	属正常结论（该主体无相关记录），不是异常、不是查无，该次调用计费
评分为 -1	属正常结论（未命中该评分模型），不要当作负分参与阈值比较
code=999 持续出现	确认三要素是否为同一人的真实有效信息；该结果表示未查得相关记录，属有效结论，同样计费

任何异常排查请一并提供响应中的 head.logId，便于我方全链路定位。

6.4 联调自检清单

- ☐ 服务地址（aiszcloud.cn:8080 或 aiszcloud.com.cn:8080）、appKey、appSecret、环境匹配无误
- ☐ 已与商务确认授权书提交方式，并已通过商务渠道提交被查询人授权书（请求中不携带授权书）
- ☐ 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- ☐ name、idCard、mobile 三项均以明文传入且都不为空
- ☐ 使用校验位合法的真实身份证号（勿用随意编造的 18 位号码），手机号为 11 位真实号码
- ☐ 若额外传了本接口不需要的字段，已把它们计入加签范围
- ☐ 能正确解析 head.errorCode 与 body.code 两级状态
- ☐ 已知悉 999 查无结果同样计费，业务侧计费预估按有效查询次数而非查得数测算
- ☐ 已实现对 result.range 字符串的二次解析（结果为 JSON 对象）与字段容错
- ☐ 已理解因子输出的是「区间档位」而非原始数值，并按 3.1.6 对照表解释档位
- ☐ 已正确处理三个特殊取值："" 缺失、"-1" 负数/评分未命中、"0" 原值为 0
- ☐ 已理解「因子大量为空、评分 -1 也是有效结论且计费」，风控策略不因此重复查询
- ☐ 已按「存在即解析、缺失即忽略」处理因子字段（预留因子集合变化）
- ☐ 已实现超时重试（依赖幂等，不重复计费）

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 sign，切勿泄露或随请求传输
logId	全链路追踪 ID (= head.logId)，排障/对账唯一凭据
uid	交易流水号 (= body.uid)，对账用
range	履约行为结果的 JSON 字符串，需二次解析（结构见 3.1.5）
区间档位	因子的输出形态：用档位号代表一个数值区间，而非原始数值（见 3.1.6）
xyp_cpl*	业务含义明确的履约行为汇总因子
xyp_t01* ~ xyp_t04*	编码型分层履约明细因子，原样透出供建模使用
xyp_model_score_*	履约风险评分， [350,950]， -1 表示未命中模型
有效查询	产出了确定结论的查询 (body.code = 001 或 999)，即本服务的计费单位
授权书	被查询人签署的个人信息查询授权文件；不通过接口传输，由调用方经商务渠道提交我方处理